

Session Management & Handoff Patterns

1. Why session management matters

Every Claude agentic loop runs inside a context window — a fixed-size memory that holds everything: the original goal, every tool call, every result, every reasoning step. In long-running sessions this window fills up. When it does, older content gets truncated from the start — and you lose your original goal, early decisions, and critical context. Session management is the discipline of keeping the right information alive across a long session without overflowing the context window.

2. Goal drift — the silent killer

Goal drift happens when an agent loses its original goal due to context overflow and starts completing tasks that no longer align with what was asked. It is dangerous because the agent does not know it is drifting — it keeps running confidently, producing wrong outputs with no error signal.

Three signs of goal drift in production:

- Agent completes successfully but the output does not match the original request.
- Agent makes decisions that contradict early constraints it can no longer see.
- Agent loops back to steps it already completed because it forgot they were done.

The fix is always preventive

- Compress and pin the goal BEFORE the window fills, not after.
- At 95% capacity, truncation has already happened — the goal is already gone.
- Compress at ~70% context usage. That is the exam rule.

3. The three tools for managing context limits

Summarise & Compress

Condenses completed steps within the same agent. Use when context is approaching its limit mid-session.

- Monitor token count after each step.
- When usage crosses ~70%, trigger a compression pass.
- Ask Claude to compress all completed steps into a brief state summary: what was done, what was found, what decisions were made.
- Drop raw tool call outputs — keep only distilled insights.
- Pin the original goal and hard constraints at the top — never let these be compressed away.
- Resume the loop with: [pinned goal] + [compressed summary] + [new steps].

Handoff Pattern

Passes compressed state to a fresh agent with a clean context window. Use at planned stage transitions in multi-agent pipelines.

- Agent 1 packages its state into a structured handoff document.
- Agent 2 receives the handoff and continues from where Agent 1 left off.
- Agent 2 starts with a fresh context window — no truncation risk.
- Every handoff package must contain: original goal (verbatim), completed steps summary, remaining work list, key decisions made.
- Handoff ≠ compression: compression continues within one agent; handoff passes the baton to a new agent.

Checkpointing
<i>Saves full state snapshot to external storage. Use for long-running tasks, before irreversible actions, and crash recovery.</i>
• Save after completing each major stage. • Save before any irreversible action. • Save when context usage crosses ~50%. • Save after recovering from an error. • Checkpoint contents: original goal, steps completed (summary), current working outputs, remaining task list, discovered constraints. • Enables resumability — if the agent crashes, restart from the last checkpoint, not from scratch.

4. Comparison — the three context management tools

Tool	What it does	When to use	Stays in same agent?
Summarise & compress	Condenses completed steps in place	Context approaching limit mid-session	Yes
Handoff	Passes state to a fresh agent	Planned stage transitions, multi-agent	No — new agent
Checkpoint	Saves snapshot to external storage	Long tasks, before irreversible actions, crash recovery	Yes — but resumable

5. What must always be pinned — never compressed away

The exam will ask what belongs in a handoff package or compressed state. There are four non-negotiables and everything else is compressible:

1	Original goal — verbatim Never paraphrase or summarise. The exact wording matters. This is what grounds all downstream decisions.
2	Hard constraints Any rules discovered during execution (e.g. 'do not modify archived records'). These must survive every compression.

3**Remaining work list**

What still needs to be done. Without this, the next agent repeats completed steps or misses remaining ones.

4**Key decisions made**

Choices that affect downstream steps. If Agent 2 doesn't know Agent 1 chose approach X, it may choose the incompatible approach Y.

Everything else is compressible

- Raw tool call outputs — distil into key findings only.
- Intermediate reasoning steps — keep conclusions, drop the working.
- Verbose API responses — extract only the relevant fields.
- Repeated or redundant context — remove duplicates.

6. Quick recall — 3 things to remember

1**Pin the original goal, always.**

It must survive every compression and handoff. Goal drift kills long-running agents silently — the agent thinks it succeeded.

2**Compress at ~70% context usage, not at 100%.**

At 100% the truncation has already happened. You need to act before content is lost, not after.

3**Handoff $\neq$ compression.**

Compression continues within one agent. Handoff passes the baton to a fresh agent with a clean window. Both solve context limits, differently.

7. Practice questions & answers

All three questions were answered correctly today — perfect score! Review the explanations to reinforce the concepts before the Week 1 Friday quiz.

Q1. A Claude agent is running a long document analysis task. After 45 steps, the context window is at 95% capacity. The agent has not yet compressed its state. What has most likely already happened at this point?

- A) The agent has automatically saved a checkpoint and can resume safely
- ✓ **B) Early content including the original goal has already been truncated from the context window**
- C) The agent will pause and request human confirmation before continuing
- D) The remaining 5% of context is sufficient to complete most tasks

Explanation:

- B is correct: At 95% capacity, truncation has already happened. The context window fills from the bottom — new content pushes in at the end, and the oldest content (original goal, early decisions) gets silently dropped from the top. This is why the rule is to compress at ~70%, not 90% or 100%.
- A is wrong: Claude does not automatically save checkpoints. Checkpointing must be explicitly designed into the system architecture.
- C is wrong: Claude does not automatically pause at context limits. It continues running silently, drifting, with no warning to the user.
- D is wrong: 5% is almost never enough to complete a meaningful task, and the damage — losing the original goal — has already been done.

Q2. You are designing a handoff package for a multi-agent research pipeline. Agent 1 has completed 20 steps of a 40-step analysis task and needs to hand off to Agent 2. Which of the following should be EXCLUDED from the handoff package?

- A) The original research goal verbatim
- ✓ **B) The full raw output of every tool call Agent 1 made across all 20 steps**
- C) A summary of key findings and decisions made so far
- D) The list of remaining steps Agent 2 still needs to complete

Explanation:

- B is correct: Raw tool call outputs are compressible content — verbose and repetitive. Including 20 full raw tool outputs would balloon Agent 2's context immediately, defeating the entire purpose of the handoff. Include only distilled insights.
- A is wrong: The original goal is non-negotiable. It must be in the handoff package verbatim, never paraphrased or summarised away.
- C is wrong: Key findings and decisions are exactly what should be in the handoff — this is the distilled value from Agent 1's work.
- D is wrong: The remaining work list is critical — without it, Agent 2 has no idea what still needs to be done and may repeat completed steps or miss remaining ones.

Q3. A Claude agent completes a long analysis task successfully. The user reviews the output and notices it doesn't match what they originally asked for — the agent analysed the wrong metric and wrote conclusions about a different region than requested. What failure mode does this describe, and what is the root cause?

- A) Prompt injection — a malicious tool result overwrote the agent's instructions mid-session
- B) Granularity error — the task was decomposed into subtasks that were too coarse
- ✓ C) **Goal drift — the original goal was lost due to context overflow and the agent continued without it**
- D) Handoff failure — the agent passed incorrect state to a subagent which misunderstood the task

Explanation:

- C is correct: This is textbook goal drift. The agent completed with no errors, but the output doesn't match the original request. The original goal (correct metric and region) was silently truncated mid-session. The agent kept running confidently, producing wrong outputs with no error signal. Goal drift is dangerous precisely because it is invisible.
- A is wrong: Prompt injection produces unexpected actions (deleting files, exfiltrating data), not a quiet drift in analysis focus.
- B is wrong: Granularity errors produce coordination overhead or context overflow problems, not a silent drift in what is being analysed.
- D is wrong: There is no mention of a handoff or subagent in the scenario. This was a single agent session that ran to completion.

Day 4 scorecard

3 / 3

Perfect score!

Context truncation timing	✓ Correct
Handoff package contents	✓ Correct
Goal drift identification	✓ Correct

Week 1 summary

Day	Topic	Score	Review?
Day 1	Agentic loops	3 / 3 ■	—

Day 2	Multi-agent patterns	2 / 3	Inherent vs implementation risks
Day 3	Task decomposition	2 / 3	Hybrid pattern identification
Day 4	Session management & handoff	3 / 3 ■	—

Next up: Day 5

Friday quiz — 10-question mixed review of the entire agentic architecture domain (27% of exam).